

Google Career Blueprint — Day 1

Detailed Interview Questions & Model Answers

Day 1 • 10 September 2026

Track	Day 1 focus
DSA	Arrays, Hash Maps, Big-O, Two Sum, Contains Duplicate, Valid Anagram
Python	Classes, self, type hints, exceptions, custom exceptions
Engineering	Document Service, Repository Pattern, Service Layer, unit testing
FastAPI / REST	API basics, HTTP methods/status codes, health endpoints
Project Deep Dive	Architecture, scalability, failure handling, async, monitoring

How to use: Answer aloud before reading the model answer. For coding questions, write the solution from memory and explain time/space complexity. For project questions, answer as if defending the design to a Google interviewer.

1. DSA — Arrays, Hash Maps & Big-O

Q1. What is an array?

Model answer: An array is an indexed sequence of elements. In a conventional contiguous array, the address of an element can be calculated directly from its index, giving constant-time random access. Python's list behaves like a dynamic array.

Q2. Why is array access $O(1)$?

Model answer: With the starting address and element size, the address of index i can be calculated directly. The work does not grow with n .

Q3. Access vs search in an array?

Model answer: Known-index access is $O(1)$. Searching for a value in an unsorted array is $O(n)$ worst case. A sorted array can use binary search in $O(\log n)$.

Q4. Complexity of inserting into the middle of an array?

Model answer: Typically $O(n)$, because later elements may need to shift. Appending to a dynamic array is amortized $O(1)$.

Q5. Array vs linked list?

Model answer: Use arrays/dynamic arrays for fast indexed access and cache-friendly traversal. Linked lists can support insertion/deletion near a known node efficiently, but random access is $O(n)$ and pointer overhead is significant.

Q6. What is a Hash Map?

Model answer: A hash map stores key-value pairs and uses a hash function to locate entries. Lookup, insertion and deletion are $O(1)$ on average, with possible worst-case degradation.

Q7. Why is Hash Map lookup average $O(1)$?

Model answer: Hashing normally identifies a bucket or table position directly. With good hashing, controlled load factor and collision handling, the expected work remains constant.

Q8. Can Hash Map lookup be $O(n)$?

Model answer: Yes. Pathological collisions or probing can cause many entries to be examined. Implementations mitigate this through collision handling and resizing.

Q9. What is a collision?

Model answer: A collision occurs when different keys map to the same hash-table location. Common strategies include chaining and open addressing.

Q10. Hash Map vs Hash Set?

Model answer: A map stores key-value pairs; a set stores unique values. Use a set for membership/seen checks and a map when you need associated information such as an index or count.

Q11. What is Big-O notation?

Model answer: Big-O describes how an algorithm's resource usage grows as input size grows, focusing on dominant growth rather than constants.

Q12. Explain $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Model answer: $O(1)$ constant; $O(\log n)$ repeatedly reduces the problem size; $O(n)$ linear scan; $O(n \log n)$ common in efficient comparison sorting; $O(n^2)$ often comes from pairwise comparisons or nested loops.

Q13. List membership vs set membership?

Model answer: x in list is $O(n)$ worst case. x in set is $O(1)$ average and $O(n)$ in pathological worst cases.

2. DSA — Core Problems

Q14. Solve Two Sum. Optimal approach?

Model answer: Use a hash map. For each x , compute complement = target - x . If complement is present, return its index and the current index; otherwise store $x \rightarrow$ index. Time $O(n)$ average; space $O(n)$.

Q15. Why is brute-force Two Sum $O(n^2)$?

Model answer: It checks every pair, giving roughly $n(n-1)/2$ comparisons, which is quadratic.

Q16. Why store value $\rightarrow$ index in Two Sum?

Model answer: The problem asks for indices. The map lets us find a previously seen complement and its original index in expected $O(1)$.

Q17. What about duplicate values in Two Sum?

Model answer: The hash-map method handles duplicates naturally. For $[3,3]$, target 6, the second 3 finds the first 3.

Q18. Two Sum without extra hash-map space?

Model answer: Sort value/index pairs and use two pointers. Sorting costs $O(n \log n)$, while preserving original indices requires carrying them with each value.

Q19. Solve Contains Duplicate.

Model answer: Scan with a set. If a value is already present, return True; otherwise add it. Time $O(n)$ average; space $O(n)$.

Q20. Why is a set natural for Contains Duplicate?

Model answer: The question is whether a value has appeared before. A set models that directly and provides average $O(1)$ membership.

Q21. Solve Valid Anagram.

Model answer: Compare character frequencies using a map, or increment counts for one string and decrement for the other. Time $O(n)$ average; space $O(k)$, k = distinct characters.

Q22. Sorting vs frequency map for Valid Anagram?

Model answer: Frequency counting is $O(n)$ average and directly expresses the requirement. Sorting is simpler in some cases but costs $O(n \log n)$.

Q23. What pattern connects the three problems?

Model answer: Two Sum: complement lookup. Contains Duplicate: seen-set membership. Valid Anagram: frequency counting. All use hashing to replace repeated linear searches with expected constant-time lookup.

3. Python — Classes, Type Hints & Exceptions

Q24. What is a class?

Model answer: A class defines data and behavior for objects. An instance is a concrete runtime object created from the class.

Q25. What is an object/instance?

Model answer: A runtime instance containing state and supporting the behavior defined by its class. A Document instance can hold id, name, type, version and status.

Q26. What is self?

Model answer: self is the conventional first parameter of an instance method and refers to the instance on which the method is invoked.

Q27. What does __init__ do?

Model answer: It initializes an instance after creation, commonly assigning constructor arguments and establishing valid initial state.

Q28. Class vs dictionary — why use a class for Document?

Model answer: A class gives a clearer domain model, encapsulates behavior and works naturally with type hints. Dictionaries are flexible but can become loosely structured.

Q29. What are type hints?

Model answer: Annotations such as name: str or documents: list[Document] that improve readability, IDE support, static analysis and maintainability.

Q30. Do type hints enforce types at runtime?

Model answer: No. Python normally does not enforce annotations automatically. Tools such as mypy/pyright provide static checking; runtime validation must be added separately.

Q31. What does Optional[str] mean?

Model answer: The value may be a string or None. Modern Python can write str | None.

Q32. What is an exception?

Model answer: An exception represents an abnormal runtime condition. It allows a precise failure to propagate to a caller that can handle it.

Q33. Why not catch every Exception everywhere?

Model answer: Broad catches can hide bugs and make diagnosis difficult. Catch specific exceptions where possible; broad handling is sometimes appropriate at an application boundary for logging and controlled responses.

Q34. raise vs try/except?

Model answer: raise creates or propagates an exception. try/except catches it so a known failure can be handled.

Q35. What is a custom exception?

Model answer: A domain-specific exception derived from Exception, such as DocumentNotFoundError. It gives callers a precise failure type.

Q36. Why custom exceptions in Document Service?

Model answer: They make business failures explicit and keep domain logic independent of HTTP. FastAPI can later translate DocumentNotFoundError into HTTP 404.

4. Python Engineering — Document Service

Q37. Purpose of Document class?

Model answer: It models the domain entity with fields such as id, name, document_type, version and status.

Q38. What is the Repository Pattern?

Model answer: It abstracts persistence operations behind methods such as create, get, list and delete, allowing the service to remain independent of storage technology.

Q39. Why not put database code directly in DocumentService?

Model answer: That couples business logic to a storage implementation and makes testing and future database changes harder.

Q40. Why use an in-memory dictionary on Day 1?

Model answer: It keeps the exercise focused on Python and architecture. The dictionary is a simple repository implementation that can later be replaced with PostgreSQL.

Q41. How would you replace it with PostgreSQL?

Model answer: Keep the repository contract stable and implement a PostgreSQL-backed repository. Add connection management, transactions, migrations, indexes and persistence error handling.

Q42. What is the Service Layer?

Model answer: It contains business use cases and rules. `create_document` can validate inputs, construct a domain object and call the repository.

Q43. Where should validation happen?

Model answer: API/schema validation handles request shape and types. The service should enforce business invariants because it can be called by non-HTTP clients. The database should also enforce persistence constraints.

Q44. What if `get_document` cannot find a document?

Model answer: Raise `DocumentNotFoundError`. The FastAPI layer can translate that domain exception into HTTP 404.

Q45. How would you test `DocumentService`?

Model answer: Use a fake/in-memory repository and test create, get, list, delete, invalid input and missing-document behavior.

5. Testing — pytest & Quality

Q46. What is a unit test?

Model answer: A test of a small unit of behavior in isolation. It should be fast, deterministic and easy to diagnose.

Q47. What makes a good unit test?

Model answer: One clear behavior, controlled inputs, meaningful assertions, deterministic results and minimal dependence on external systems.

Q48. What should Day-1 tests cover?

Model answer: Create; retrieve; nonexistent retrieval raises `DocumentNotFoundError`; delete then retrieval fails; empty name raises `InvalidDocumentError`. Also test list and invalid document types.

Q49. Why test exceptions explicitly?

Model answer: Exceptions are part of the contract. Testing them proves invalid or missing data produces the intended failure instead of an ambiguous result.

Q50. What is mocking?

Model answer: Replacing a dependency with a controlled test double. Use it when you need to isolate or verify interactions; prefer simple fakes when they make tests clearer.

6. FastAPI & REST

Q51. What is FastAPI?

Model answer: A Python web framework for APIs using type annotations and Pydantic-based validation, async support and automatic OpenAPI documentation.

Q52. Why FastAPI for this project?

Model answer: It fits the Python/AI ecosystem and provides productive API development, schema validation, documentation and async support.

Q53. What is REST?

Model answer: An architectural style where resources are exposed through representations and HTTP methods express operations, with stateless interactions as a core principle.

Q54. Why POST for creating a document?

Model answer: POST is appropriate when a client asks a collection such as /documents to create/process a new resource and the server can generate its identifier.

Q55. GET vs POST?

Model answer: GET retrieves data and is safe/idempotent. POST submits data for creation or processing and is not inherently idempotent.

Q56. Successful creation status?

Model answer: Typically 201 Created, optionally with the created representation and a Location header.

Q57. What does 404 mean?

Model answer: The requested resource was not found. A missing document is a natural 404 case.

Q58. 400 vs 422?

Model answer: 400 is a general bad-request response. 422 Unprocessable Content is commonly used by FastAPI for schema validation failures.

Q59. Why /health?

Model answer: It gives an operational signal that the service is responsive. Production systems often distinguish liveness and readiness.

Q60. Liveness vs readiness?

Model answer: Liveness asks whether the process is alive; readiness asks whether the instance is ready to receive traffic.

7. API Design & Production Thinking

Q61. Design a Document API.

Model answer: POST /documents; GET /documents/{id}; GET /documents; DELETE /documents/{id}. Later add pagination, filtering, authentication/authorization, versioning, request IDs and consistent error schemas.

Q62. Why pagination?

Model answer: Unbounded responses can consume excessive memory, bandwidth and latency. Pagination bounds response size; cursor pagination is often better for large changing datasets.

Q63. How prevent unlimited uploads?

Model answer: Use authentication, request/body limits, quotas, rate limits, file-type/content validation and storage limits. Stream large uploads instead of loading them fully into memory.

Q64. What if PostgreSQL is down?

Model answer: Fail predictably. Use timeouts, bounded retries where safe, structured logs and alerts. Never report success when persistence failed.

Q65. How would you scale the API?

Model answer: Keep API instances stateless, load-balance them, externalize state to PostgreSQL/Redis/object storage, containerize and scale horizontally. Then address database and connection-pool bottlenecks.

Q66. When use async in FastAPI?

Model answer: For I/O-bound work with async-capable libraries. Async does not make CPU-heavy work faster; CPU-heavy tasks may need workers or multiprocessing.

Q67. How monitor the service?

Model answer: Use structured logs, latency/error/throughput metrics, resource metrics and traces. For future AI features, add retrieval quality, model quality, token/cost, drift and agent failure metrics.

8. Project 1 — Google-Style Deep Dive

Q68. Walk through the architecture.

Model answer: Client → FastAPI API layer → service/business layer → repository/data layer → PostgreSQL/Redis/object storage as needed. Later, ingestion feeds retrieval; agents use controlled tools; evaluation and observability capture quality and operational signals.

Q69. Why separate API, service and repository?

Model answer: API handles transport/schema concerns; service handles business rules; repository handles persistence. The separation improves testability and replaceability.

Q70. Most important Day-1 design decision?

Model answer: Build a thin vertical slice with clean boundaries instead of jumping into advanced AI. A document domain and API establish foundations for later RAG and agent capabilities.

Q71. Why not start with an LLM chatbot?

Model answer: A chatbot alone demonstrates integration more than engineering depth. The platform is intended to demonstrate architecture, APIs, data, retrieval, agents, evaluation and production reliability.

Q72. How add RAG later?

Model answer: Ingest and normalize documents, chunk them, create embeddings, retrieve using dense or hybrid BM25+vector search, rerank, apply metadata filters, and generate answers with citations. Evaluate retrieval using Recall@K, MRR and nDCG.

Q73. How prevent unsafe agent actions?

Model answer: Use least-privilege tools, explicit permissions, schema validation, sandboxing where needed, audit logs, human approval for high-impact actions, rate limits and rollback.

Q74. How evaluate an AI/RAG system?

Model answer: Retrieval: Recall@K, MRR, nDCG. Generation: faithfulness, relevance, citation correctness. Agents: task success, tool selection, steps, failure rate, latency and cost. Maintain a regression dataset.

Q75. How handle a failing downstream service?

Model answer: Use timeouts, retry classification, bounded exponential backoff, circuit breakers where appropriate, safe fallbacks and clear observability. Avoid uncontrolled retries that amplify outages.

Q76. Is this really production-ready?

Model answer: Be honest. State what is implemented, tested and measured, then identify remaining hardening such as persistent storage, auth, observability, deployment, load testing and failure testing.

9. Behavioral / Engineering Judgment

Q77. Tell me about something you struggled with on Day 1.

Model answer: Use a real example: explain the problem, your initial assumption, investigation, fix and lesson. Demonstrate debugging and learning rather than claiming everything worked immediately.

Q78. Why are you building this project?

Model answer: Connect it to the target competencies: software fundamentals, DSA, Python, backend APIs, ML/AI systems, production infrastructure, system design and measurable quality.

Q79. How decide what not to build?

Model answer: Prioritize features that prove required competency and support the next architectural step. Defer UI polish and advanced AI until the core workflow is correct, tested and measurable.

Q80. How explain a technical trade-off to a non-expert?

Model answer: State the decision, problem, alternatives, trade-off and expected impact, using a simple example and avoiding unnecessary jargon.

Day 1 — 20 Core Questions to Master

1. What is an array, and why is indexed access $O(1)$?
2. What is a Hash Map, and why is lookup $O(1)$ on average?
3. Hash Map vs Hash Set — when do you use each?
4. Explain $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, and $O(n^2)$.
5. Solve Two Sum and explain the hash-map approach.
6. What is the time/space complexity of Two Sum?
7. Solve Contains Duplicate using a set.
8. Solve Valid Anagram using frequency counting.
9. What is a Python class and what is an instance?
10. What is self and why is it used in instance methods?
11. What are type hints?
12. Do Python type hints enforce types at runtime?
13. What is an exception and when should you catch one?
14. What is a custom exception and why use one?
15. What is the Repository Pattern?
16. Service Layer vs Repository — what belongs where?
17. What makes a unit test good?
18. Why FastAPI for this project?
19. Why POST /documents for creation?
20. Walk through the complete Project 1 architecture and justify the separation of concerns.

Day 1 Answer Framework

1. Definition — one clear sentence.
2. Mechanism — explain how it works.
3. Complexity / trade-off — state time/space or the key trade-off.
4. Example — give a small Day-1 example.
5. Production angle — when relevant, explain testing, reliability or scalability.

Rapid-Fire Self-Test

1. Why does a set beat a list for duplicate detection?
2. Why does hashing trade memory for speed?
3. Why is $O(n^2)$ problematic as n grows?
4. Why should domain logic not know about HTTP 404?
5. Why can a type hint be violated at runtime?
6. Why is a custom exception better than returning None for a missing document?
7. Why should the service still validate if FastAPI validates requests?
8. Why is POST generally used for creation?
9. Why is healthy different from ready?
10. What would you test before calling the Day-1 slice complete?